org.apache.commons.csv.CSVParserTest:886: specialinvoke $stack4.<java.io.StringReader: void <init>(java.lang.String)>("a,b,cn1,2,3nx,y,z")

org.apache.commons.csv.CSVParserTest:886: $stack4 = new java.io.StringReader

org.apache.commons.csv.ExtendedBufferedReader:55: specialinvoke this.<java.io.BufferedReader: void <init>(java.io.Reader)>(reader)

org.apache.commons.csv.CSVParser:283: specialinvoke $stack10.<org.apache.commons.csv.ExtendedBufferedReader: void <init>(java.io.Reader)>(reader)

org.apache.commons.csv.CSVParser:283: $stack10 = new org.apache.commons.csv.ExtendedBufferedReader

org.apache.commons.csv.CSVParser:252: specialinvoke this.<org.apache.commons.csv.CSVParser: void <init>(java.io.Reader,org.apache.commons.csv.CSVFormat,long,long)>(reader, format, 0L, 1L)

org.apache.commons.csv.CSVFormat:431: if header != null

org.apache.commons.csv.CSVFormat:1286: specialinvoke $stack2.<org.apache.commons.csv.CSVFormat: void <init>(char,java.lang.Character,org.apache.commons.csv.QuoteMode,java.lang.Character,java.lang.Character,boolean,boolean,java.lang.String,java.lang.String,java.lang.Object[],java.lang.String[],boolean,boolean,boolean)>($stack15, $stack14, $stack13, $stack12, $stack11, $stack10, $stack9, $stack8, $stack7, $stack6, $stack5, skipHeaderRecord, $stack4, $stack3)

org.apache.commons.csv.CSVFormat:1286: $stack5 = this.<org.apache.commons.csv.CSVFormat: java.lang.String[] header>

org.apache.commons.csv.CSVFormat:966: specialinvoke $stack2.<org.apache.commons.csv.CSVFormat: void <init>(char,java.lang.Character,org.apache.commons.csv.QuoteMode,java.lang.Character,java.lang.Character,boolean,boolean,java.lang.String,java.lang.String,java.lang.Object[],java.lang.String[],boolean,boolean,boolean)>($stack15, $stack14, $stack13, $stack12, $stack11, $stack10, $stack9, $stack8, $stack7, $stack6, header, $stack5, $stack4, $stack3) org.apache.commons.csv.CSVParserTest:887: $stack5[0] = "X" org.apache.commons.csv.CSVParserTest:887: $stack5 = newarray (java.lang.String)[3] org.apache.commons.csv.CSVParserTest:887: $stack5[2] = "Z" org.apache.commons.csv.CSVParserTest:887: $stack5[1] = "Y"

org.apache.commons.csv.CSVFormat:1273: $stack1 = virtualinvoke this.<org.apache.commons.csv.CSVFormat: org.apache.commons.csv.CSVFormat withSkipHeaderRecord(boolean)>(1)

org.apache.commons.csv.CSVFormat:431: this.<org.apache.commons.csv.CSVFormat: java.lang.String[] header> = $stack17

org.apache.commons.csv.CSVParserTest:887: $stack7 = virtualinvoke $stack6.<org.apache.commons.csv.CSVFormat: org.apache.commons.csv.CSVFormat withHeader(java.lang.String[])>($stack5)

org.apache.commons.csv.CSVFormat:421: this.<org.apache.commons.csv.CSVFormat: java.lang.Character quoteCharacter> = quoteChar

org.apache.commons.csv.CSVFormat:1286: $stack14 = this.<org.apache.commons.csv.CSVFormat: java.lang.Character quoteCharacter>

org.apache.commons.csv.CSVFormat:966: $stack14 = this.<org.apache.commons.csv.CSVFormat: java.lang.Character quoteCharacter>

org.apache.commons.csv.CSVFormat:229: specialinvoke $stack0.<org.apache.commons.csv.CSVFormat: void <init>(char,java.lang.Character,org.apache.commons.csv.QuoteMode,java.lang.Character,java.lang.Character,boolean,boolean,java.lang.String,java.lang.String,java.lang.Object[],java.lang.String[],boolean,boolean,boolean)>(44, $stack1, null, null, null, 0, 1, "rn", null, null, null, 0, 0, 0) org.apache.commons.csv.CSVFormat:229: $stack1 = <org.apache.commons.csv.Constants: java.lang.Character DOUBLE_QUOTE_CHAR>

org.apache.commons.csv.Constants:36: <org.apache.commons.csv.Constants: java.lang.Character DOUBLE_QUOTE_CHAR> = $stack0

org.apache.commons.csv.Constants:36: $stack0 = staticinvoke <java.lang.Character: java.lang.Character valueOf(char)>(34)

org.apache.commons.csv.CSVFormat:429: this.<org.apache.commons.csv.CSVFormat: java.lang.String nullString> = nullString

org.apache.commons.csv.CSVFormat:966: $stack7 = this.<org.apache.commons.csv.CSVFormat: java.lang.String nullString> org.apache.commons.csv.CSVFormat:1286: $stack7 = this.<org.apache.commons.csv.CSVFormat: java.lang.String nullString>

org.apache.commons.csv.CSVFormat:420: this.<org.apache.commons.csv.CSVFormat: char delimiter> = delimiter

org.apache.commons.csv.CSVFormat:966: $stack15 = this.<org.apache.commons.csv.CSVFormat: char delimiter> org.apache.commons.csv.CSVFormat:1286: $stack15 = this.<org.apache.commons.csv.CSVFormat: char delimiter>

org.apache.commons.csv.CSVFormat:424: this.<org.apache.commons.csv.CSVFormat: java.lang.Character escapeCharacter> = escape

org.apache.commons.csv.CSVFormat:966: $stack11 = this.<org.apache.commons.csv.CSVFormat: java.lang.Character escapeCharacter> org.apache.commons.csv.CSVFormat:1286: $stack11 = this.<org.apache.commons.csv.CSVFormat: java.lang.Character escapeCharacter>

org.apache.commons.csv.CSVFormat:433: this.<org.apache.commons.csv.CSVFormat: boolean ignoreHeaderCase> = ignoreHeaderCase

org.apache.commons.csv.CSVFormat:966: $stack3 = this.<org.apache.commons.csv.CSVFormat: boolean ignoreHeaderCase> org.apache.commons.csv.CSVFormat:1286: $stack3 = this.<org.apache.commons.csv.CSVFormat: boolean ignoreHeaderCase>

org.apache.commons.csv.CSVFormat:427: this.<org.apache.commons.csv.CSVFormat: boolean ignoreEmptyLines> = ignoreEmptyLines

org.apache.commons.csv.CSVFormat:966: $stack9 = this.<org.apache.commons.csv.CSVFormat: boolean ignoreEmptyLines> org.apache.commons.csv.CSVFormat:1286: $stack9 = this.<org.apache.commons.csv.CSVFormat: boolean ignoreEmptyLines>

org.apache.commons.csv.CSVFormat:431: $stack17 = (java.lang.String[]) $stack16

org.apache.commons.csv.CSVFormat:431: $stack16 = virtualinvoke header.<java.lang.Object: java.lang.Object clone()>()

org.apache.commons.csv.CSVParserTest:887: $stack8 = virtualinvoke $stack7.<org.apache.commons.csv.CSVFormat: org.apache.commons.csv.CSVFormat withSkipHeaderRecord()>()

org.apache.commons.csv.CSVFormat:432: this.<org.apache.commons.csv.CSVFormat: boolean skipHeaderRecord> = skipHeaderRecord

org.apache.commons.csv.CSVFormat:551: $stack1 = this.<org.apache.commons.csv.CSVFormat: char delimiter>

org.apache.commons.csv.Lexer:63: $stack3 = virtualinvoke format.<org.apache.commons.csv.CSVFormat: char getDelimiter()>()

org.apache.commons.csv.CSVParser:283: specialinvoke $stack9.<org.apache.commons.csv.Lexer: void <init>(org.apache.commons.csv.CSVFormat,org.apache.commons.csv.ExtendedBufferedReader)>(format, $stack10)

org.apache.commons.csv.CSVFormat:641: $stack1 = this.<org.apache.commons.csv.CSVFormat: java.lang.Character quoteCharacter>

org.apache.commons.csv.Lexer:65: $stack6 = virtualinvoke format.<org.apache.commons.csv.CSVFormat: java.lang.Character getQuoteCharacter()>()

org.apache.commons.csv.CSVFormat:560: $stack1 = this.<org.apache.commons.csv.CSVFormat: java.lang.Character escapeCharacter>

org.apache.commons.csv.Lexer:64: $stack4 = virtualinvoke format.<org.apache.commons.csv.CSVFormat: java.lang.Character getEscapeCharacter()>()

org.apache.commons.csv.CSVFormat:598: $stack1 = this.<org.apache.commons.csv.CSVFormat: boolean ignoreEmptyLines>

org.apache.commons.csv.Lexer:68: $stack11 = virtualinvoke format.<org.apache.commons.csv.CSVFormat: boolean getIgnoreEmptyLines()>()

org.apache.commons.csv.CSVFormat:632: $stack1 = this.<org.apache.commons.csv.CSVFormat: java.lang.String nullString>

org.apache.commons.csv.CSVParser:291: $stack7 = virtualinvoke $stack6.<org.apache.commons.csv.CSVFormat: java.lang.String getNullString()>()

org.apache.commons.csv.CSVParser:503: specialinvoke this.<org.apache.commons.csv.CSVParser: void addRecordValue()>() org.apache.commons.csv.CSVParser:500: specialinvoke this.<org.apache.commons.csv.CSVParser: void addRecordValue()>()

org.apache.commons.csv.CSVFormat:569: $stack1 = this.<org.apache.commons.csv.CSVFormat: java.lang.String[] header>

org.apache.commons.csv.CSVParser:376: $stack9 = virtualinvoke $stack8.<org.apache.commons.csv.CSVFormat: java.lang.String[] getHeader()>()

org.apache.commons.csv.CSVParser:284: $stack11 = specialinvoke this.<org.apache.commons.csv.CSVParser: java.util.Map initializeHeader()>()

org.apache.commons.csv.CSVFormat:668: $stack1 = this.<org.apache.commons.csv.CSVFormat: boolean skipHeaderRecord> org.apache.commons.csv.CSVFormat:569: $stack2 = this.<org.apache.commons.csv.CSVFormat: java.lang.String[] header>

org.apache.commons.csv.CSVParser:390: $stack15 = virtualinvoke $stack14.<org.apache.commons.csv.CSVFormat: boolean getSkipHeaderRecord()>()

org.apache.commons.csv.CSVFormat:616: $stack1 = this.<org.apache.commons.csv.CSVFormat: boolean ignoreHeaderCase>

org.apache.commons.csv.CSVParser:378: $stack11 = virtualinvoke $stack10.<org.apache.commons.csv.CSVFormat: boolean getIgnoreHeaderCase()>()

org.apache.commons.csv.CSVFormat:743: specialinvoke $stack2.<org.apache.commons.csv.CSVParser: void <init>(java.io.Reader,org.apache.commons.csv.CSVFormat)>(in, this)

org.apache.commons.csv.CSVParserTest:887: $stack9 = virtualinvoke $stack8.<org.apache.commons.csv.CSVFormat: org.apache.commons.csv.CSVParser parse(java.io.Reader)>($stack4)

org.apache.commons.csv.CSVFormat:743: $stack2 = new org.apache.commons.csv.CSVParser

org.apache.commons.csv.Lexer:62: this.<org.apache.commons.csv.Lexer: org.apache.commons.csv.ExtendedBufferedReader reader> = reader

org.apache.commons.csv.ExtendedBufferedReader:41: this.<org.apache.commons.csv.ExtendedBufferedReader: int lastChar> = $i0

org.apache.commons.csv.ExtendedBufferedReader:-1: $i0 = (int) -2

org.apache.commons.csv.CSVParser:232: specialinvoke $stack8.<org.apache.commons.csv.Token: void <init>()>()

org.apache.commons.csv.CSVParser:284: this.<org.apache.commons.csv.CSVParser: java.util.Map headerMap> = $stack11

org.apache.commons.csv.CSVParser:411: return hdrMap

org.apache.commons.csv.CSVParser:407: interfaceinvoke hdrMap.<java.util.Map: java.lang.Object put(java.lang.Object,java.lang.Object)>(header, $stack19)

org.apache.commons.csv.CSVParser:378: hdrMap = $stack12

org.apache.commons.csv.CSVParser:219: this.<org.apache.commons.csv.CSVParser: java.util.List record> = $stack7

org.apache.commons.csv.CSVParser:219: specialinvoke $stack7.<java.util.ArrayList: void <init>()>()

org.apache.commons.csv.CSVParser:219: $stack7 = new java.util.ArrayList

org.apache.commons.csv.CSVParser:286: this.<org.apache.commons.csv.CSVParser: long recordNumber> = $stack12

org.apache.commons.csv.CSVParser:285: this.<org.apache.commons.csv.CSVParser: long characterOffset> = characterOffset org.apache.commons.csv.CSVParser:286: $stack12 = recordNumber - 1L

org.apache.commons.csv.CSVParser:492: $stack6 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.CSVParser$1:438: $stack3 = virtualinvoke $stack2.<org.apache.commons.csv.CSVParser: org.apache.commons.csv.CSVRecord nextRecord()>()

org.apache.commons.csv.CSVParser:292: interfaceinvoke $stack8.<java.util.List: boolean add(java.lang.Object)>($stack11)

org.apache.commons.csv.CSVParser:391: virtualinvoke this.<org.apache.commons.csv.CSVParser: org.apache.commons.csv.CSVRecord nextRecord()>()

org.apache.commons.csv.CSVParser$1:467: $stack4 = specialinvoke this.<org.apache.commons.csv.CSVParser$1: org.apache.commons.csv.CSVRecord getNextRecord()>()

org.apache.commons.csv.CSVParser:292: $stack8 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.CSVParser:292: $stack11 = $stack5

org.apache.commons.csv.CSVParser:390: if $stack15 == 0

org.apache.commons.csv.Token:55: this.<org.apache.commons.csv.Token: java.lang.StringBuilder content> = $stack2

org.apache.commons.csv.Token:55: specialinvoke $stack2.<java.lang.StringBuilder: void <init>(int)>(50)

org.apache.commons.csv.Token:55: $stack2 = new java.lang.StringBuilder

org.apache.commons.csv.Token$Type:36: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type INVALID> = $stack0

org.apache.commons.csv.Token$Type:34: specialinvoke this.<java.lang.Enum: void <init>(java.lang.String,int)>(l1, l2)

org.apache.commons.csv.Token$Type:36: $stack0 = new org.apache.commons.csv.Token$Type org.apache.commons.csv.Token$Type:36: specialinvoke $stack0.<org.apache.commons.csv.Token$Type: void <init>(java.lang.String,int)>("INVALID", 0) org.apache.commons.csv.Token$Type:39: specialinvoke $stack1.<org.apache.commons.csv.Token$Type: void <init>(java.lang.String,int)>("TOKEN", 1) org.apache.commons.csv.Token$Type:45: specialinvoke $stack3.<org.apache.commons.csv.Token$Type: void <init>(java.lang.String,int)>("EORECORD", 3) org.apache.commons.csv.Token$Type:42: specialinvoke $stack2.<org.apache.commons.csv.Token$Type: void <init>(java.lang.String,int)>("EOF", 2) org.apache.commons.csv.Token$Type:48: specialinvoke $stack4.<org.apache.commons.csv.Token$Type: void <init>(java.lang.String,int)>("COMMENT", 4) org.apache.commons.csv.Token$Type:42: $stack2 = new org.apache.commons.csv.Token$Type org.apache.commons.csv.Token$Type:48: $stack4 = new org.apache.commons.csv.Token$Type org.apache.commons.csv.Token$Type:45: $stack3 = new org.apache.commons.csv.Token$Type org.apache.commons.csv.Token$Type:39: $stack1 = new org.apache.commons.csv.Token$Type

org.apache.commons.csv.Token$Type:34: return

org.apache.commons.csv.Token:62: $stack2 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type INVALID>

org.apache.commons.csv.CSVParser:496: virtualinvoke $stack10.<org.apache.commons.csv.Token: void reset()>()

org.apache.commons.csv.Lexer:133: $stack17 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type INVALID>

org.apache.commons.csv.CSVParser:497: virtualinvoke $stack12.<org.apache.commons.csv.Lexer: org.apache.commons.csv.Token nextToken(org.apache.commons.csv.Token)>($stack11)

org.apache.commons.csv.Token$Type:34: $stack6 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type INVALID>

org.apache.commons.csv.Token$Type:39: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type TOKEN> = $stack1

org.apache.commons.csv.Lexer:197: $stack29 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type TOKEN>

org.apache.commons.csv.Lexer:161: specialinvoke this.<org.apache.commons.csv.Lexer: org.apache.commons.csv.Token parseSimpleToken(org.apache.commons.csv.Token,int)>(token, c)

org.apache.commons.csv.Lexer:153: if $stack22 == 0

org.apache.commons.csv.Token$Type:34: $stack7 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type TOKEN>

org.apache.commons.csv.Token$Type:42: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EOF> = $stack2

org.apache.commons.csv.Token$Type:34: $stack8 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EOF>

org.apache.commons.csv.Token$Type:45: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EORECORD> = $stack3

org.apache.commons.csv.Lexer:190: $stack35 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EORECORD>

org.apache.commons.csv.Token$Type:34: $stack9 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type EORECORD>

org.apache.commons.csv.Token$Type:48: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type COMMENT> = $stack4

org.apache.commons.csv.Token$Type:34: $stack10 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type COMMENT>

org.apache.commons.csv.Token$Type:34: <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type[] $VALUES> = $stack5

org.apache.commons.csv.Token$Type:34: $stack5[0] = $stack6 org.apache.commons.csv.Token$Type:34: $stack5 = newarray (org.apache.commons.csv.Token$Type)[5] org.apache.commons.csv.Token$Type:34: $stack5[2] = $stack8 org.apache.commons.csv.Token$Type:34: $stack5[4] = $stack10 org.apache.commons.csv.Token$Type:34: $stack5[3] = $stack9 org.apache.commons.csv.Token$Type:34: $stack5[1] = $stack7

org.apache.commons.csv.Token$Type:34: $stack0 = <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type[] $VALUES>

org.apache.commons.csv.CSVParser$2:498: $stack1 = staticinvoke <org.apache.commons.csv.Token$Type: org.apache.commons.csv.Token$Type[] values()>()

org.apache.commons.csv.Token:61: $stack1 = this.<org.apache.commons.csv.Token: java.lang.StringBuilder content>

org.apache.commons.csv.Lexer:208: virtualinvoke $stack9.<java.lang.StringBuilder: java.lang.StringBuilder append(char)>($stack8)

org.apache.commons.csv.Lexer:208: $stack8 = (char) ch org.apache.commons.csv.Lexer:208: $stack9 = token.<org.apache.commons.csv.Token: java.lang.StringBuilder content>

org.apache.commons.csv.Lexer:199: if $stack7 == 0

org.apache.commons.csv.ExtendedBufferedReader:56: return

org.apache.commons.csv.ExtendedBufferedReader:78: $stack1 = this.<org.apache.commons.csv.ExtendedBufferedReader: int lastChar>

org.apache.commons.csv.ExtendedBufferedReader:64: this.<org.apache.commons.csv.ExtendedBufferedReader: int lastChar> = $stack2 org.apache.commons.csv.Lexer:86: lastChar = virtualinvoke $stack7.<org.apache.commons.csv.ExtendedBufferedReader: int getLastChar()>()

org.apache.commons.csv.Lexer:89: $stack10 = virtualinvoke $stack9.<org.apache.commons.csv.ExtendedBufferedReader: int read()>()

org.apache.commons.csv.ExtendedBufferedReader:60: $stack2 = specialinvoke this.<java.io.BufferedReader: int read()>()

org.apache.commons.csv.Lexer:209: ch = virtualinvoke $stack11.<org.apache.commons.csv.ExtendedBufferedReader: int read()>()

org.apache.commons.csv.Lexer:65: $stack7 = specialinvoke this.<org.apache.commons.csv.Lexer: char mapNullToDisabled(java.lang.Character)>($stack6)

org.apache.commons.csv.Lexer:68: this.<org.apache.commons.csv.Lexer: boolean ignoreEmptyLines> = $stack11

org.apache.commons.csv.CSVFormat:598: return $stack1

org.apache.commons.csv.Lexer:63: this.<org.apache.commons.csv.Lexer: char delimiter> = $stack3

org.apache.commons.csv.CSVFormat:551: return $stack1

org.apache.commons.csv.Lexer:65: this.<org.apache.commons.csv.Lexer: char quoteChar> = $stack7

org.apache.commons.csv.Lexer:292: return $stack2

org.apache.commons.csv.Lexer:64: $stack5 = specialinvoke this.<org.apache.commons.csv.Lexer: char mapNullToDisabled(java.lang.Character)>($stack4)

org.apache.commons.csv.Lexer:292: $stack2 = 65534 org.apache.commons.csv.Lexer:292: $stack2 = virtualinvoke c.<java.lang.Character: char charValue()>()

org.apache.commons.csv.Lexer:64: this.<org.apache.commons.csv.Lexer: char escape> = $stack5

org.apache.commons.csv.Lexer:209: $stack11 = this.<org.apache.commons.csv.Lexer: org.apache.commons.csv.ExtendedBufferedReader reader>

org.apache.commons.csv.Lexer:89: $stack9 = this.<org.apache.commons.csv.Lexer: org.apache.commons.csv.ExtendedBufferedReader reader>

org.apache.commons.csv.Lexer:410: $stack2 = this.<org.apache.commons.csv.Lexer: char delimiter>

org.apache.commons.csv.Lexer:196: $stack6 = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean isDelimiter(int)>(ch) org.apache.commons.csv.Lexer:143: $stack20 = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean isDelimiter(int)>(c) org.apache.commons.csv.Lexer:113: $stack15 = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean isDelimiter(int)>(lastChar)

org.apache.commons.csv.Lexer:113: if $stack13 != 0

org.apache.commons.csv.CSVFormat:560: return $stack1

org.apache.commons.csv.Lexer:292: if c != null

org.apache.commons.csv.CSVFormat:641: return $stack1

org.apache.commons.csv.Lexer:414: $stack2 = this.<org.apache.commons.csv.Lexer: char escape>

org.apache.commons.csv.Lexer:199: $stack7 = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean isEscape(int)>(ch)

org.apache.commons.csv.Lexer:418: $stack2 = this.<org.apache.commons.csv.Lexer: char quoteChar>

org.apache.commons.csv.Lexer:150: $stack21 = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean isQuoteChar(int)>(c)

org.apache.commons.csv.Lexer:97: $stack12 = this.<org.apache.commons.csv.Lexer: boolean ignoreEmptyLines>

org.apache.commons.csv.CSVParser:407: $stack19 = staticinvoke <java.lang.Integer: java.lang.Integer valueOf(int)>(nextRecord)

org.apache.commons.csv.CSVParser:400: $stack18 = interfaceinvoke hdrMap.<java.util.Map: boolean containsKey(java.lang.Object)>(header) org.apache.commons.csv.CSVParser:401: $stack40 = virtualinvoke header.<java.lang.String: java.lang.String trim()>()

org.apache.commons.csv.CSVParser:399: header = headerRecord[nextRecord]

org.apache.commons.csv.CSVParser:398: if nextRecord >= $stack17

org.apache.commons.csv.CSVParser:393: headerRecord = $stack9

org.apache.commons.csv.CSVParser:398: nextRecord = nextRecord + 1

org.apache.commons.csv.CSVParser:398: $stack17 = lengthof headerRecord

org.apache.commons.csv.CSVParser:398: nextRecord = 0

org.apache.commons.csv.CSVParser:397: if headerRecord == null

org.apache.commons.csv.CSVFormat:569: return $stack4

org.apache.commons.csv.CSVParser:378: $u1 = new java.util.LinkedHashMap

org.apache.commons.csv.CSVParser:378: if $stack11 == 0

org.apache.commons.csv.CSVFormat:616: return $stack1

org.apache.commons.csv.CSVParser:377: if $stack9 == null

org.apache.commons.csv.CSVFormat:569: $stack4 = (java.lang.String[]) $stack3

org.apache.commons.csv.CSVFormat:668: return $stack1

org.apache.commons.csv.CSVParser:-1: $stack12 = $u1

org.apache.commons.csv.CSVFormat:569: if $stack1 == null org.apache.commons.csv.CSVFormat:569: $stack3 = virtualinvoke $stack2.<java.lang.Object: java.lang.Object clone()>()

org.apache.commons.csv.CSVParser:498: $stack18 = $stack16[$stack17]

org.apache.commons.csv.CSVParser:498: $stack17 = virtualinvoke $stack15.<org.apache.commons.csv.Token$Type: int ordinal()>()

org.apache.commons.csv.CSVParser:498: $stack16 = <org.apache.commons.csv.CSVParser$2: int[] $SwitchMap$org$apache$commons$csv$Token$Type>

org.apache.commons.csv.CSVParser:498: $stack15 = $stack14.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type>

org.apache.commons.csv.CSVParser$2:498: <org.apache.commons.csv.CSVParser$2: int[] $SwitchMap$org$apache$commons$csv$Token$Type> = $stack3

org.apache.commons.csv.Lexer:190: token.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type> = $stack35 org.apache.commons.csv.Lexer:197: token.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type> = $stack29

org.apache.commons.csv.Lexer:189: if $stack4 == 0

org.apache.commons.csv.Lexer:196: if $stack6 == 0

org.apache.commons.csv.CSVParser:498: tableswitch($stack18) { case 1:

org.apache.commons.csv.CSVParser$2:498: $stack3 = newarray (int)[$stack2]

org.apache.commons.csv.CSVParser:492: interfaceinvoke $stack6.<java.util.List: void clear()>()

org.apache.commons.csv.Token:62: this.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type> = $stack2 org.apache.commons.csv.Token:61: virtualinvoke $stack1.<java.lang.StringBuilder: void setLength(int)>(0)

org.apache.commons.csv.Lexer:133: $stack18 = token.<org.apache.commons.csv.Token: org.apache.commons.csv.Token$Type type>

org.apache.commons.csv.Lexer:406: return $stack2

org.apache.commons.csv.Lexer:192: $stack5 = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean isEndOfFile(int)>(ch) org.apache.commons.csv.Lexer:153: $stack22 = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean isEndOfFile(int)>(c) org.apache.commons.csv.Lexer:113: $stack13 = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean isEndOfFile(int)>(lastChar) org.apache.commons.csv.Lexer:97: if $stack12 == 0

org.apache.commons.csv.Lexer:406: $stack2 = 0

org.apache.commons.csv.Lexer:119: $stack16 = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean isStartOfLine(int)>(lastChar)

org.apache.commons.csv.Lexer:98: if eol == 0

org.apache.commons.csv.Lexer:94: eol = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean readEndOfLine(int)>($stack10)

org.apache.commons.csv.Lexer:133: if $stack18 != $stack17

org.apache.commons.csv.Lexer:119: if $stack16 == 0

org.apache.commons.csv.Lexer:113: if $stack15 != 0

org.apache.commons.csv.Lexer:399: return $stack2

org.apache.commons.csv.Lexer:89: c = $stack10

org.apache.commons.csv.ExtendedBufferedReader:66: return $stack7

org.apache.commons.csv.ExtendedBufferedReader:66: $stack7 = this.<org.apache.commons.csv.ExtendedBufferedReader: int lastChar>

org.apache.commons.csv.ExtendedBufferedReader:78: return $stack1

org.apache.commons.csv.Lexer:399: if ch == 10

org.apache.commons.csv.ExtendedBufferedReader:65: $stack6 = $stack5 + 1L

org.apache.commons.csv.ExtendedBufferedReader:65: $stack5 = this.<org.apache.commons.csv.ExtendedBufferedReader: long position>

org.apache.commons.csv.ExtendedBufferedReader:65: this.<org.apache.commons.csv.ExtendedBufferedReader: long position> = $stack6

org.apache.commons.csv.Lexer:378: if ch != 13

org.apache.commons.csv.Lexer:378: if ch == 10

org.apache.commons.csv.Lexer:189: $stack4 = virtualinvoke this.<org.apache.commons.csv.Lexer: boolean readEndOfLine(int)>(ch)

org.apache.commons.csv.Lexer:378: return $stack2

org.apache.commons.csv.Lexer:378: $stack2 = 1 org.apache.commons.csv.Lexer:378: $stack2 = 0

org.apache.commons.csv.Lexer:146: if eol == 0

org.apache.commons.csv.Lexer:143: if $stack20 == 0

org.apache.commons.csv.Lexer:410: return $stack3

org.apache.commons.csv.Lexer:406: if ch != $i0

org.apache.commons.csv.Lexer:-1: $i0 = (int) -1

org.apache.commons.csv.Lexer:410: if ch != $i0

org.apache.commons.csv.Lexer:-1: $i0 = (int) $stack2

org.apache.commons.csv.Lexer:410: $stack3 = 1 org.apache.commons.csv.Lexer:410: $stack3 = 0

org.apache.commons.csv.Lexer:399: $stack2 = 0

org.apache.commons.csv.Lexer:399: $stack2 = 1

org.apache.commons.csv.Lexer:150: if $stack21 == 0

org.apache.commons.csv.Lexer:418: return $stack3

org.apache.commons.csv.Lexer:418: $stack3 = 0

org.apache.commons.csv.Lexer:418: if ch != $i0

org.apache.commons.csv.Lexer:414: return $stack3 org.apache.commons.csv.Lexer:192: if $stack5 == 0

org.apache.commons.csv.Lexer:414: $stack3 = 0

org.apache.commons.csv.Lexer:414: if ch != $i0

org.apache.commons.csv.CSVParser:290: $stack4 = $stack3.<org.apache.commons.csv.Token: java.lang.StringBuilder content>

org.apache.commons.csv.Token$Type:34: return $stack2

org.apache.commons.csv.Token$Type:34: $stack2 = (org.apache.commons.csv.Token$Type[]) $stack1

org.apache.commons.csv.Token$Type:34: $stack1 = virtualinvoke $stack0.<java.lang.Object: java.lang.Object clone()>()

org.apache.commons.csv.CSVParser$2:498: $stack2 = lengthof $stack1

org.apache.commons.csv.CSVParser:292: if $stack9 == 0

org.apache.commons.csv.CSVParser:292: $stack9 = virtualinvoke $stack5.<java.lang.String: boolean equals(java.lang.Object)>($stack7)

org.apache.commons.csv.CSVParser:290: $stack5 = virtualinvoke $stack4.<java.lang.StringBuilder: java.lang.String toString()>()

org.apache.commons.csv.CSVFormat:632: return $stack1

org.apache.commons.csv.ExtendedBufferedReader:172: $stack1 = this.<org.apache.commons.csv.ExtendedBufferedReader: long position>

org.apache.commons.csv.Lexer:310: $stack2 = virtualinvoke $stack1.<org.apache.commons.csv.ExtendedBufferedReader: long getPosition()>()

org.apache.commons.csv.CSVParser:494: $stack8 = virtualinvoke $stack7.<org.apache.commons.csv.Lexer: long getCharacterPosition()>()

org.apache.commons.csv.CSVParser:529: $stack44 = this.<org.apache.commons.csv.CSVParser: java.util.Map headerMap>

org.apache.commons.csv.CSVParser:494: $stack9 = this.<org.apache.commons.csv.CSVParser: long characterOffset> org.apache.commons.csv.CSVParser:527: $stack34 = this.<org.apache.commons.csv.CSVParser: long recordNumber>

org.apache.commons.csv.CSVParser$1:433: return $stack1

org.apache.commons.csv.CSVParser$1:473: return next

org.apache.commons.csv.CSVParserTest:889: $stack11 = interfaceinvoke $stack10.<java.util.Iterator: java.lang.Object next()>()

org.apache.commons.csv.CSVParser$1:433: $stack1 = virtualinvoke this.<org.apache.commons.csv.CSVParser$1: org.apache.commons.csv.CSVRecord next()>()

org.apache.commons.csv.CSVParser$1:467: next = $stack4

org.apache.commons.csv.CSVParserTest:892: $stack17 = virtualinvoke record.<org.apache.commons.csv.CSVRecord: java.lang.String get(java.lang.String)>("Z")

org.apache.commons.csv.CSVParserTest:889: record = (org.apache.commons.csv.CSVRecord) $stack11 org.apache.commons.csv.CSVRecord:102: $stack6 = interfaceinvoke $stack5.<java.util.Map: java.lang.Object get(java.lang.Object)>(name)

org.apache.commons.csv.CSVRecord:102: $stack5 = this.<org.apache.commons.csv.CSVRecord: java.util.Map mapping>

org.apache.commons.csv.CSVParserTest:891: $stack15 = virtualinvoke record.<org.apache.commons.csv.CSVRecord: java.lang.String get(java.lang.String)>("Y") org.apache.commons.csv.CSVParserTest:890: $stack13 = virtualinvoke record.<org.apache.commons.csv.CSVRecord: java.lang.String get(java.lang.String)>("X")

org.apache.commons.csv.CSVParser$1:462: next = this.<org.apache.commons.csv.CSVParser$1: org.apache.commons.csv.CSVRecord current>

org.apache.commons.csv.CSVParser$1:463: this.<org.apache.commons.csv.CSVParser$1: org.apache.commons.csv.CSVRecord current> = null

org.apache.commons.csv.CSVParser$1:465: if next != null

org.apache.commons.csv.CSVParser$1:438: return $stack3

org.apache.commons.csv.CSVParser:532: return result

org.apache.commons.csv.CSVParser$1:438: $stack2 = this.<org.apache.commons.csv.CSVParser$1: org.apache.commons.csv.CSVParser this$0>

org.apache.commons.csv.CSVParser:529: result = $stack37

org.apache.commons.csv.CSVRecord:58: this.<org.apache.commons.csv.CSVRecord: java.lang.String comment> = comment org.apache.commons.csv.CSVRecord:59: this.<org.apache.commons.csv.CSVRecord: long characterPosition> = characterPosition

org.apache.commons.csv.CSVRecord:56: this.<org.apache.commons.csv.CSVRecord: java.lang.String[] values> = $stack8 org.apache.commons.csv.CSVRecord:57: this.<org.apache.commons.csv.CSVRecord: java.util.Map mapping> = mapping

org.apache.commons.csv.CSVRecord:55: this.<org.apache.commons.csv.CSVRecord: long recordNumber> = recordNumber

org.apache.commons.csv.CSVRecord:54: specialinvoke this.<java.lang.Object: void <init>()>()

org.apache.commons.csv.CSVParser:529: $stack37 = new org.apache.commons.csv.CSVRecord

org.apache.commons.csv.CSVParser:494: startCharPosition = $stack8 + $stack9

org.apache.commons.csv.CSVParser:529: specialinvoke $stack37.<org.apache.commons.csv.CSVRecord: void <init>(java.lang.String[],java.util.Map,java.lang.String,long,long)>($stack45, $stack44, $stack36, $stack43, startCharPosition)

org.apache.commons.csv.CSVParser:528: $stack36 = null

org.apache.commons.csv.CSVRecord:56: $stack8 = values

org.apache.commons.csv.CSVParser:529: $stack43 = this.<org.apache.commons.csv.CSVParser: long recordNumber>

org.apache.commons.csv.CSVParser:526: if $stack33 != 0

org.apache.commons.csv.CSVParser:526: $stack33 = interfaceinvoke $stack32.<java.util.List: boolean isEmpty()>()

org.apache.commons.csv.CSVParser:526: $stack32 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.Lexer:310: return $stack2

org.apache.commons.csv.ExtendedBufferedReader:172: return $stack1

org.apache.commons.csv.CSVParser:527: this.<org.apache.commons.csv.CSVParser: long recordNumber> = $stack35

org.apache.commons.csv.CSVParser:527: $stack35 = $stack34 + 1L

org.apache.commons.csv.CSVParser:493: sb = null

org.apache.commons.csv.CSVParser:529: $stack41 = newarray (java.lang.String)[$stack40]

org.apache.commons.csv.CSVParser:529: $stack40 = interfaceinvoke $stack38.<java.util.List: int size()>()

org.apache.commons.csv.CSVParser:529: $stack38 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.CSVParser:529: $stack42 = interfaceinvoke $stack39.<java.util.List: java.lang.Object[] toArray(java.lang.Object[])>($stack41)

org.apache.commons.csv.CSVParser:529: $stack39 = this.<org.apache.commons.csv.CSVParser: java.util.List record>

org.apache.commons.csv.CSVParser:529: $stack45 = (java.lang.String[]) $stack42

org.apache.commons.csv.CSVParser:528: if sb != null

org.apache.commons.csv.CSVRecord:56: if values == null org.apache.commons.csv.CSVRecord:60: return

org.apache.commons.csv.CSVParserTest:892: staticinvoke <org.junit.Assert: void assertEquals(java.lang.Object,java.lang.Object)>("3", $stack17)